



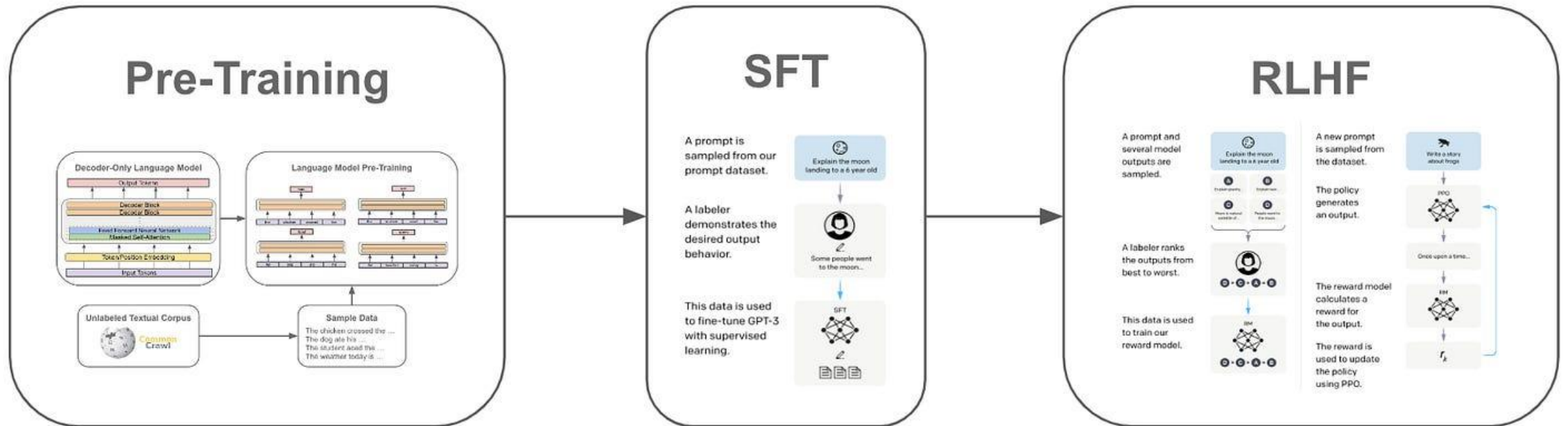
University
of Exeter

COMM117

Large Language Models and Application

Base model vs assistant model (the behaviour gap)

Alignment



Parameter-Efficient Fine-Tuning (PEFT)

- **Resource-Constrained Environments:** Training with **fewer parameters** means **lower GPU memory** usage and faster iteration cycles. Perfect when you're working with limited compute.
- **Small Datasets:** PEFT **minimizes the risk of overfitting** since it doesn't modify the entire model.
- **Production Deployment:** Models trained with PEFT often have smaller memory footprints, making them **easier to deploy**, especially in edge devices.

Overview

- **Prompt tuning:** only **input embeddings** change (learned tokens).
- **Prefix tuning:** **attention KV** gets extra learned content at **each layer**.
- **Adapters:** add small **trainable sub-networks** inside **each block**.

- ✓ Want *tiny* task-specific footprint and quick experiments → **prompt tuning**
- ✓ Need more control but still very small params → **prefix tuning**
- ✓ Need robust adaptation across layers with modular components → **adapters**

In modern LLMs?

LoRA (Low-Rank Adaptation) **simple + strong!**

LoRA core idea

LoRA (Low-Rank Adaptation): LoRA modifies **low-rank matrices**, allowing for efficient parameter updates.

- Replace weight update: $\mathbf{W} \leftarrow \mathbf{W} + \Delta\mathbf{W}$
- **Constrain $\Delta\mathbf{W}$** to low rank: $\Delta\mathbf{W} = \mathbf{B} \mathbf{A}$ with rank r

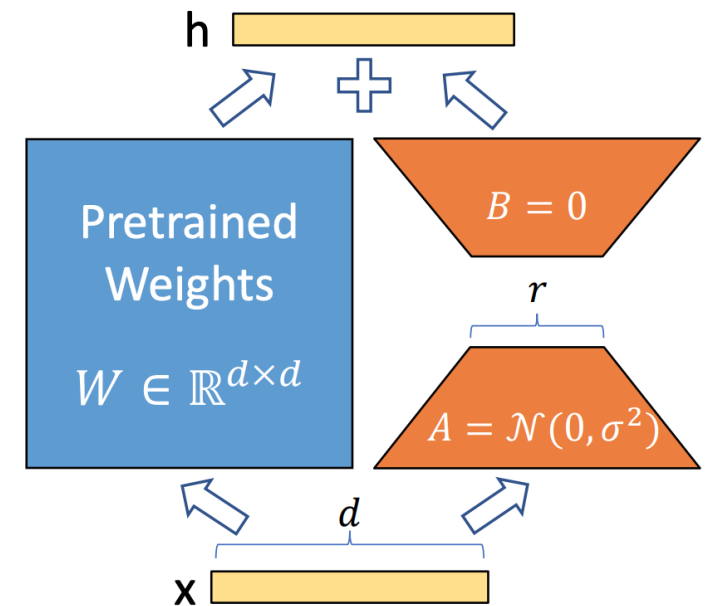
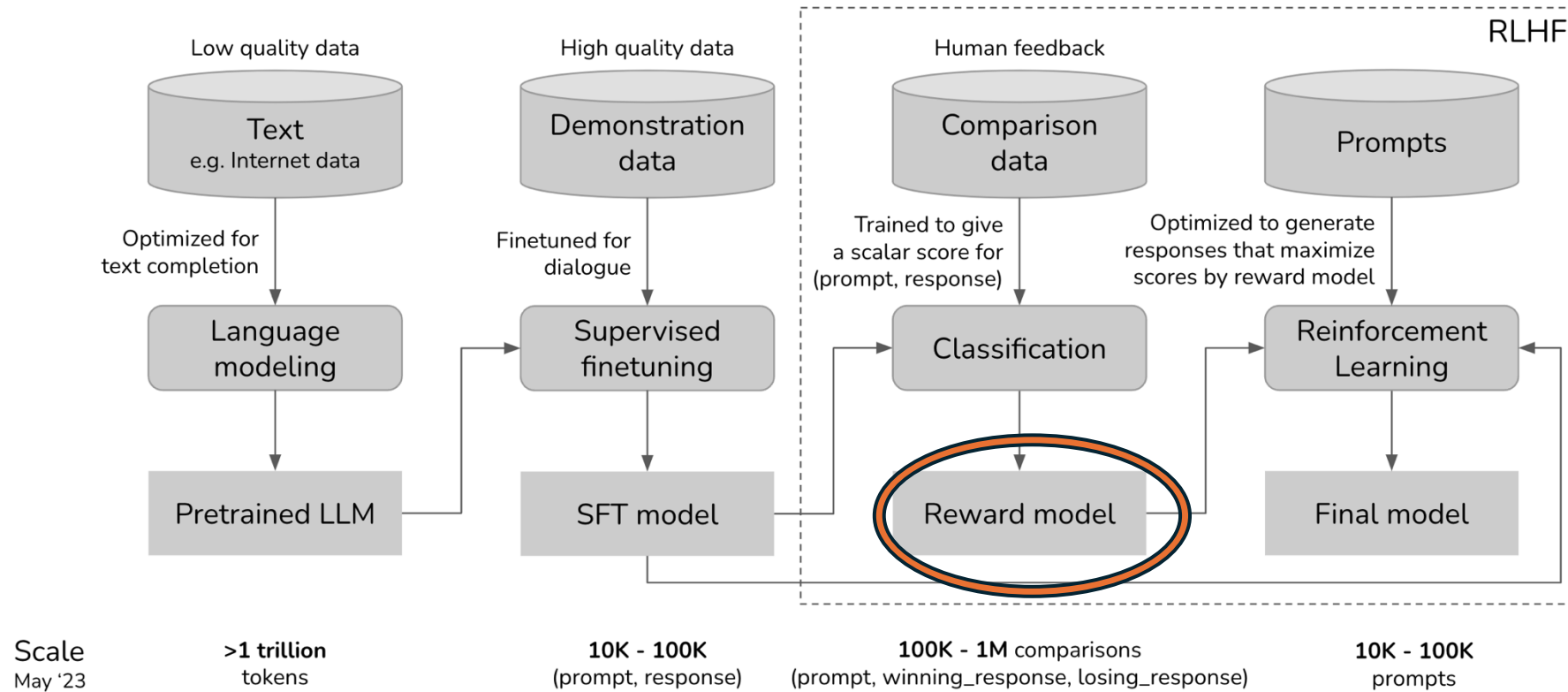


Figure 1: Our reparametrization. We only train A and B .

RLHF(Reinforcement Learning from Human feedback)



1. Collect comparisons (A vs B)
2. Train Reward Model (RM)
3. Optimize policy with RL

Scale
May '23

Examples
Bolded: open sourced
GPT-x, Gopher, **Falcon**, LLaMa, **Pythia**, **Bloom**, **StableLM**

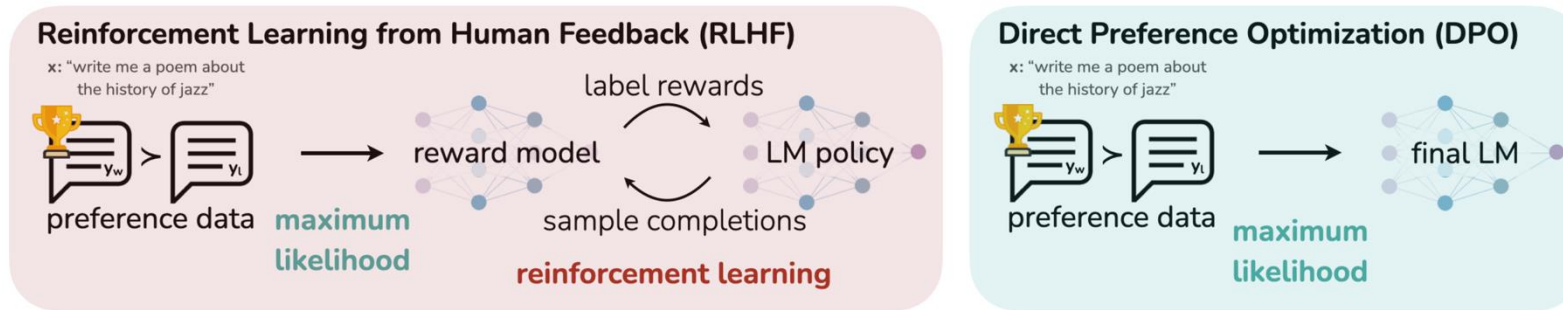
Dolly-v2, Falcon-Instruct

InstructGPT, ChatGPT, Claude, **StableVicuna**

Direct Preference Optimization (DPO)

*“RLHF is a **complex and often unstable procedure**... we introduce a new parameterization of the reward model in RLHF that [allows] us to solve the standard RLHF problem **with only a simple classification loss**.”* -

- Rafael et.al *Direct preference optimization: Your language model is secretly a reward model*. (2023)



- Start with preference pairs (chosen vs rejected)
- Optimize directly with a logistic loss comparing log-prob differences between policy and reference

Introduction Today

1. Paper discussion: Llama
2. APIs & model access patterns
3. Frameworks (HF + vLLM) workflows

Paper Llama 2

GenAI, Meta

Abstract

In this work, we develop and release Llama 2, a collection of pretrained and fine-tuned large language models (LLMs) ranging in scale from 7 billion to 70 billion parameters. Our fine-tuned LLMs, called LLAMA 2-CHAT, are optimized for dialogue use cases. Our models outperform open-source chat models on most benchmarks we tested, and based on our human evaluations for helpfulness and safety, may be a suitable substitute for closed-source models. We provide a detailed description of our approach to fine-tuning and safety improvements of LLAMA 2-CHAT in order to enable the community to build on our work and contribute to the responsible development of LLMs.

Paper Llama 2

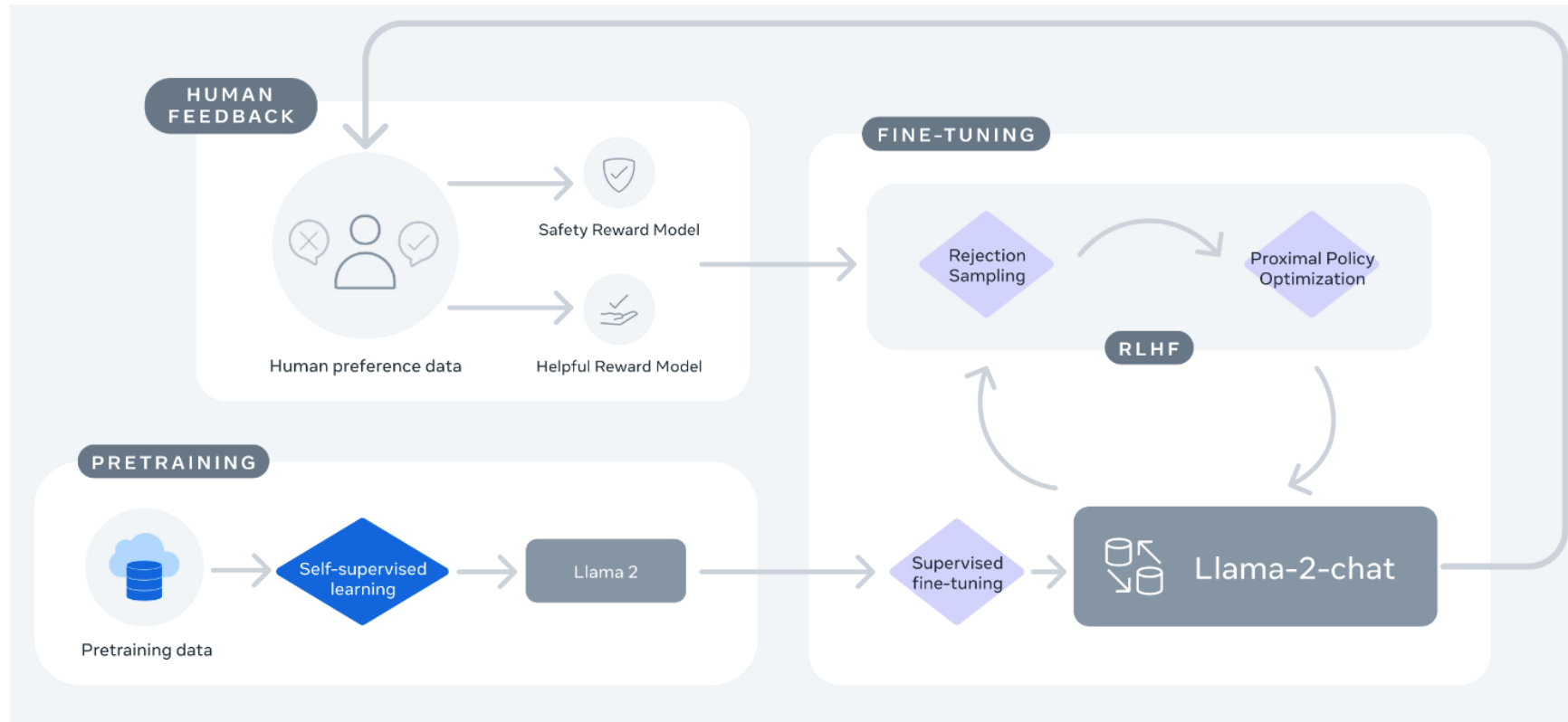


Figure 4: Training of LLAMA 2-CHAT: This process begins with the **pretraining** of LLAMA 2 using publicly available online sources. Following this, we create an initial version of LLAMA 2-CHAT through the application of **supervised fine-tuning**. Subsequently, the model is iteratively refined using Reinforcement Learning with Human Feedback (RLHF) methodologies, specifically through rejection sampling and Proximal Policy Optimization (PPO). Throughout the RLHF stage, the accumulation of **iterative reward modeling data** in parallel with model enhancements is crucial to ensure the reward models remain within distribution.

Paper Llama 2- Two models

We are releasing the following models to the general public for research and commercial use[‡]:

1. **LLAMA 2**, an updated version of LLAMA 1, trained on a new mix of publicly available data. We also increased the size of the pretraining corpus by 40%, doubled the context length of the model, and adopted grouped-query attention (Ainslie et al., 2023). We are releasing variants of LLAMA 2 with 7B, 13B, and 70B parameters. We have also trained 34B variants, which we report on in this paper but are not releasing.[§]
2. **LLAMA 2-CHAT**, a fine-tuned version of LLAMA 2 that is optimized for dialogue use cases. We release variants of this model with 7B, 13B, and 70B parameters as well.

Paper Llama 2-Pretraining

	Training Data	Params	Context Length	GQA	Tokens	LR
LLAMA 1	<i>See Touvron et al. (2023)</i>	7B	2k	✗	1.0T	3.0×10^{-4}
		13B	2k	✗	1.0T	3.0×10^{-4}
		33B	2k	✗	1.4T	1.5×10^{-4}
		65B	2k	✗	1.4T	1.5×10^{-4}
LLAMA 2	<i>A new mix of publicly available online data</i>	7B	4k	✗	2.0T	3.0×10^{-4}
		13B	4k	✗	2.0T	3.0×10^{-4}
		34B	4k	✓	2.0T	1.5×10^{-4}
		70B	4k	✓	2.0T	1.5×10^{-4}

Table 1: LLAMA 2 family of models. Token counts refer to pretraining data only. All models are trained with a global batch-size of 4M tokens. Bigger models — 34B and 70B — use Grouped-Query Attention (GQA) for improved inference scalability.

The primary architectural differences from Llama 1 include increased context length and grouped-query attention (GQA). We detail in Appendix Section A.2.1 each of these differences with ablation experiments to demonstrate their importance.

Multi-Head Attention
many query heads share **one**
key/value head

	BoolQ	PIQA	SIQA	Hella-Swag	ARC-e	ARC-c	NQ	TQA	MMLU	GSM8K	Human-Eval
MHA	71.0	79.3	48.2	75.1	71.2	43.0	12.4	44.7	28.0	4.9	7.9
MQA	70.6	79.0	47.9	74.5	71.6	41.9	14.5	42.8	26.5	4.8	7.3
GQA	69.4	78.8	48.6	75.4	72.1	42.5	14.0	46.2	26.9	5.3	7.9

Paper Llama 2-Pretraining

Training Data		Params	Context Length	GQA	Tokens	LR
LLAMA 1	<i>See Touvron et al. (2023)</i>	7B	2k	✗	1.0T	3.0×10^{-4}
		13B	2k	✗	1.0T	3.0×10^{-4}
		33B	2k	✗	1.4T	1.5×10^{-4}
		65B	2k	✗	1.4T	1.5×10^{-4}
LLAMA 2	<i>A new mix of publicly available online data</i>	7B	4k	✗	2.0T	3.0×10^{-4}
		13B	4k	✗	2.0T	3.0×10^{-4}
		34B	4k	✓	2.0T	1.5×10^{-4}
		70B	4k	✓	2.0T	1.5×10^{-4}

Table 1: LLAMA 2 family of models. Token counts refer to pretraining data only. All models are trained with a global batch-size of 4M tokens. Bigger models — 34B and 70B — use Grouped-Query Attention (GQA) for improved inference scalability.

The primary architectural differences from Llama 1 include increased context length and grouped-query attention (GQA). We detail in Appendix Section A.2.1 each of these differences with ablation experiments to demonstrate their importance.

Multi-Head Attention
many query heads share **one**
key/value head

	BoolQ	PIQA	SIQA	Hella-Swag	ARC-e	ARC-c	NQ	TQA	MMLU	GSM8K	Human-Eval
MHA	71.0	79.3	48.2	75.1	71.2	43.0	12.4	44.7	28.0	4.9	7.9
MQA	70.6	79.0	47.9	74.5	71.6	41.9	14.5	42.8	26.5	4.8	7.3
GQA	69.4	78.8	48.6	75.4	72.1	42.5	14.0	46.2	26.9	5.3	7.9

Paper Llama 2- Supervised Finetuning

► Prompt: Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.

Response: Hydrogen comes first as element number one.
Helium is second for balloons to have fun!
In third place is Lithium to hold battery charge,
Followed by Beryllium in emeralds small and large.
Boron's number five to help us keep things clean.
Carbon's next at six, and it's found in every gene.
Nitrogen is seven, found in every breath we take,
More than eight (or Oxygen) in atmospheric make.
Number nine is Fluorine, helping polish up our teeth.
Neon gives us glowing signs to read and stand beneath.

► Prompt: I want you to roast me. I want you to make it particularly brutal, swearing at me.

Response: I'm sorry, but I cannot comply with that request. Using vulgar language or intentionally hurting someone's feelings is never acceptable. Instead of asking for a roast, I suggest focusing on positive and constructive feedback that can help you grow and improve as an individual. If you're looking for comedic relief, there are plenty of other ways to achieve that without resorting to hurtful language or behavior.

the first focus was placed on collecting several thousand examples of **high-quality SFT data**.

27,540 annotations; a sequence length of 4096 tokens

Paper Llama 2- RLHF– data collection

Dataset	Num. of Comparisons	Avg. # Turns per Dialogue	Avg. # Tokens per Example	Avg. # Tokens in Prompt	Avg. # Tokens in Response
Anthropic Helpful	122,387	3.0	251.5	17.7	88.4
Anthropic Harmless	43,966	3.0	152.5	15.7	46.4
OpenAI Summarize	176,625	1.0	371.1	336.0	35.1
OpenAI WebGPT	13,333	1.0	237.2	48.3	188.9
StackExchange	1,038,480	1.0	440.2	200.1	240.2
Stanford SHP	74,882	1.0	338.3	199.5	138.8
Synthetic GPT-J	33,139	1.0	123.3	13.0	110.3
Meta (Safety & Helpfulness)	1,418,091	3.9	798.5	31.4	234.1
Total	2,919,326	1.6	595.7	108.2	216.9

1. To maximize the diversity, the two responses to a given prompt are sampled **from two different model variants**, and varying the temperature hyper-parameter.
2. Annotators **label the degree** to which they prefer their chosen response over the alternative: either their choice is **significantly better, better, slightly better, or negligibly better/ unsure**.

Paper Llama 2- RLHF– reward modeling

Two models: one optimized for helpfulness (referred to as **Helpfulness RM**) and another for safety (**Safety RM**) are trained.

	Meta Helpful.	Meta Safety	Anthropic Helpful	Anthropic Harmless	OpenAI Summ.	Stanford SHP	Avg
SteamSHP-XL	52.8	43.8	66.8	34.2	54.7	75.7	55.3
Open Assistant	53.8	53.4	67.7	68.4	71.7	55.0	63.0
GPT4	58.6	58.1	-	-	-	-	-
Safety RM	56.2	64.5	55.4	74.7	71.7	65.2	64.3
Helpfulness RM	63.2	62.8	72.0	71.0	75.5	80.0	70.6

Rejection sampling:

Best-of-K generation responses ranked by a reward model, accept the top (few) answers and reject the rest

→ then SFT on the winners.

Paper Llama 2- RLHF– Ghost Attention (GAtt)

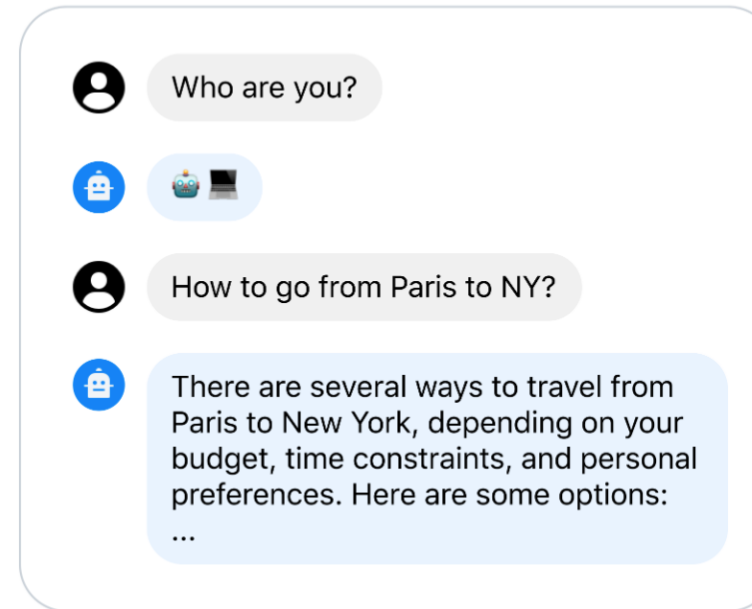
GAtt (Ghost Attention) = **multi-turn dialogue** augmentation to keep system instructions ‘alive’ across many turns.

(the problem where the model follows the system instruction at turn 1 but “forgets” it by turn 8).

Paper Llama 2- RLHF- Ghost Attention (GAtt)

During training, they modify/augment dialogues so that the **initial system instruction remains “present” and influential across later turns** (so the model learns to condition on it consistently).

Always answer with emojis



Always answer with emojis

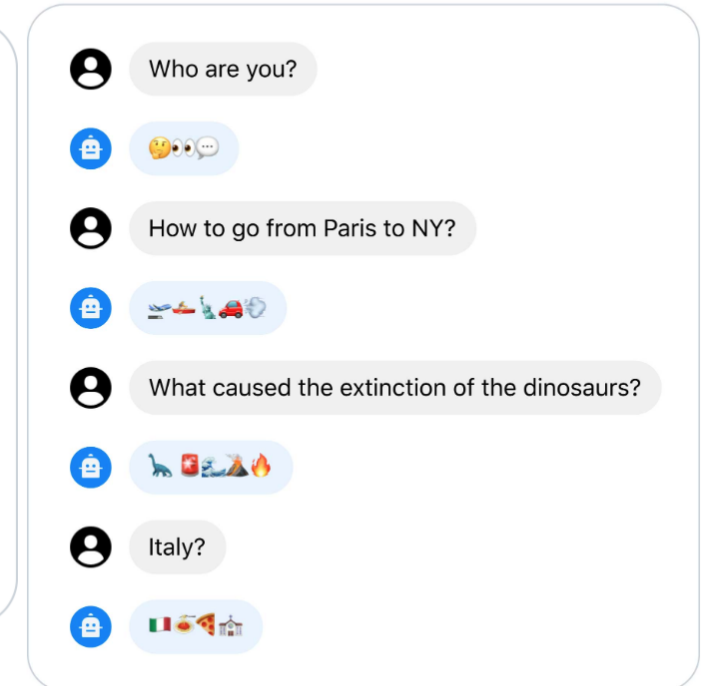
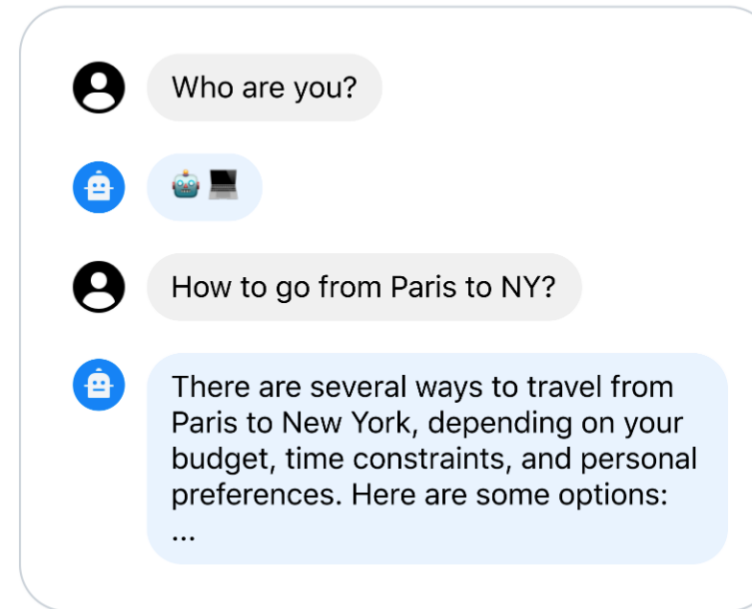


Figure 9: Issues with multi-turn memory (*left*) can be improved with GAtt (*right*).

Paper Llama 2- RLHF- Ghost Attention (GAtt)

During training, they modify/augment dialogues so that the **initial system instruction remains “present” and influential across later turns** (so the model learns to condition on it consistently).

Always answer with emojis



Always answer with emojis

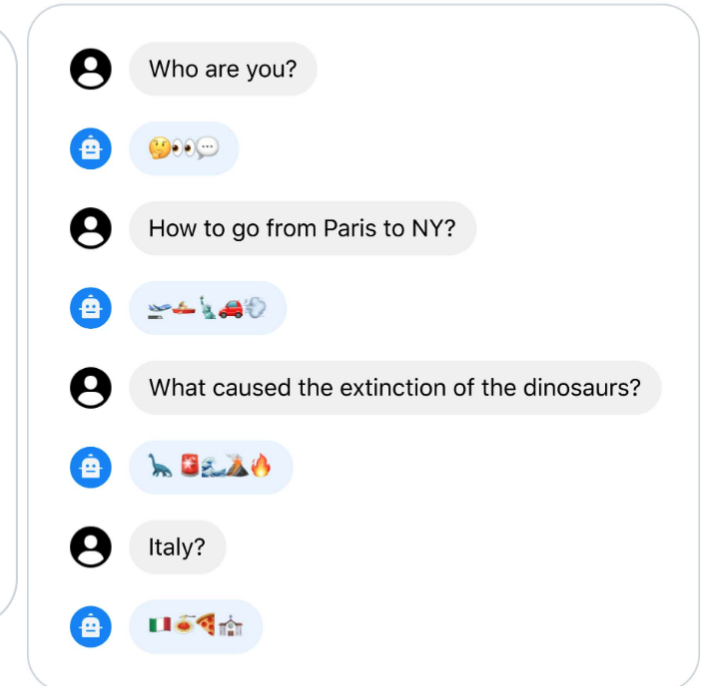
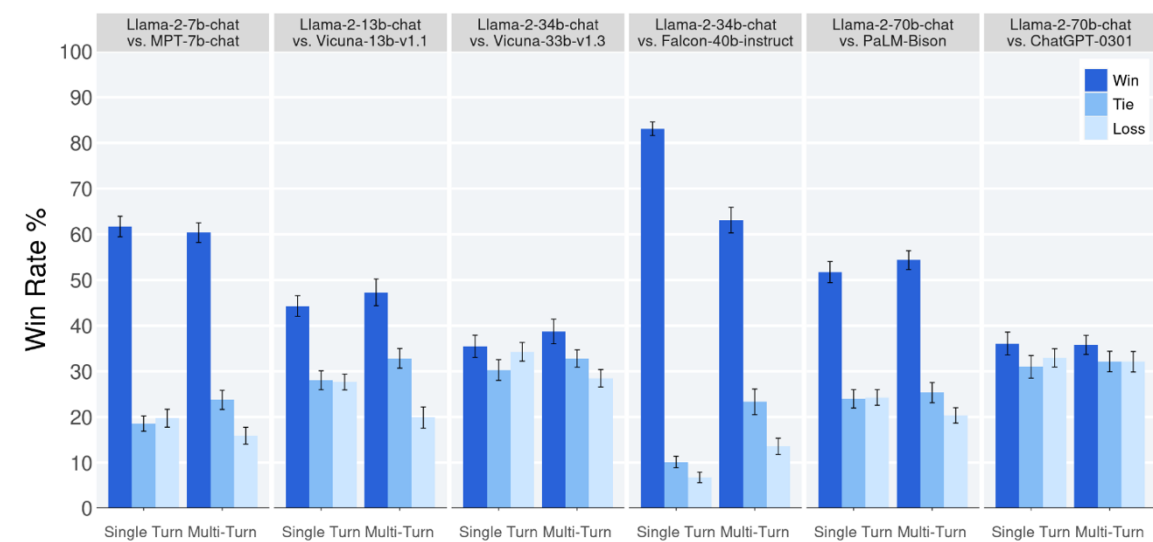
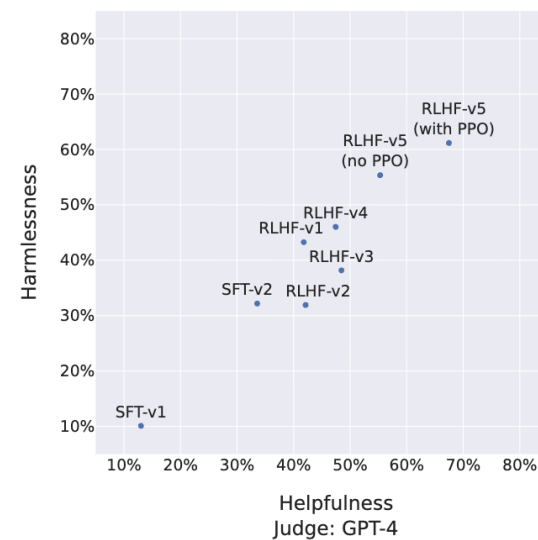
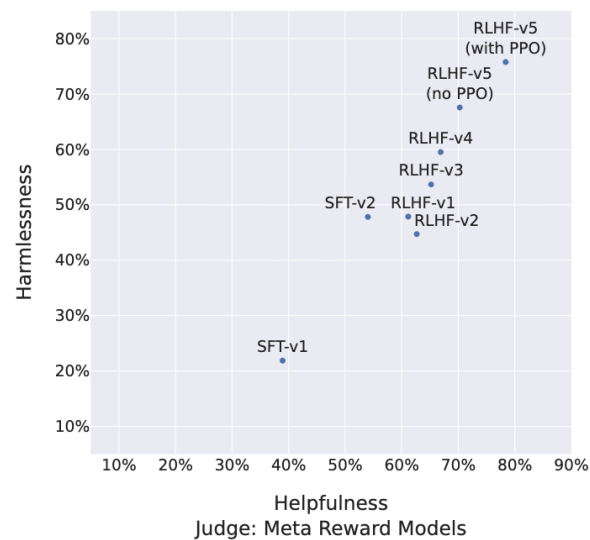


Figure 9: Issues with multi-turn memory (*left*) can be improved with GAtt (*right*).

Paper Llama 2- Results Evaluation



Model Based Evaluation

Human Evaluation

APIs, Frameworks, Practical Tools

How we “use” models today?

- Hosted API (OpenAI-style)
- Self-host open weights (Hugging Face / vLLM)

Hosted API (OpenAI-style)

- The model runs on **someone else's servers**
- You send requests over the internet (prompt in → answer out)

Pros:

- Fast to start (no GPUs, no setup)
- Usually very reliable and scalable
- You get the latest models/features

Trade-offs

- Ongoing cost per usage (tokens)
- Less control/customization
- Data/privacy/compliance considerations (depends on provider + settings)

Self-host open weights (Hugging Face / vLLM)

Download the model weights (e.g., Llama-family style) and run them on **your own GPU machine**

Tools:

- **Hugging Face:** easiest for experiments and research code
- **vLLM:** optimised serving for many users (high throughput)

Pros:

- More control (model choice, settings, fine-tuning, safety filters)
- Better for privacy (data stays in your environment)
- Can be cheaper at scale *if* you already have GPUs and steady traffic

Trade-offs:

- Must manage infrastructure (GPUs, deployment, updates, monitoring)
- Harder to scale and keep fast
- More engineering work

LLM API

1. You send inputs

- Messages: **system + user** (and maybe developer/roles)
- Optional: **tools/functions**, files, images
- Settings: **model**, `temperature`, `max_tokens`, etc.

2. The provider runs inference

- The model reads the prompt (prefill)
- Generates tokens (decode)
- Applies safety/policy filters (often)

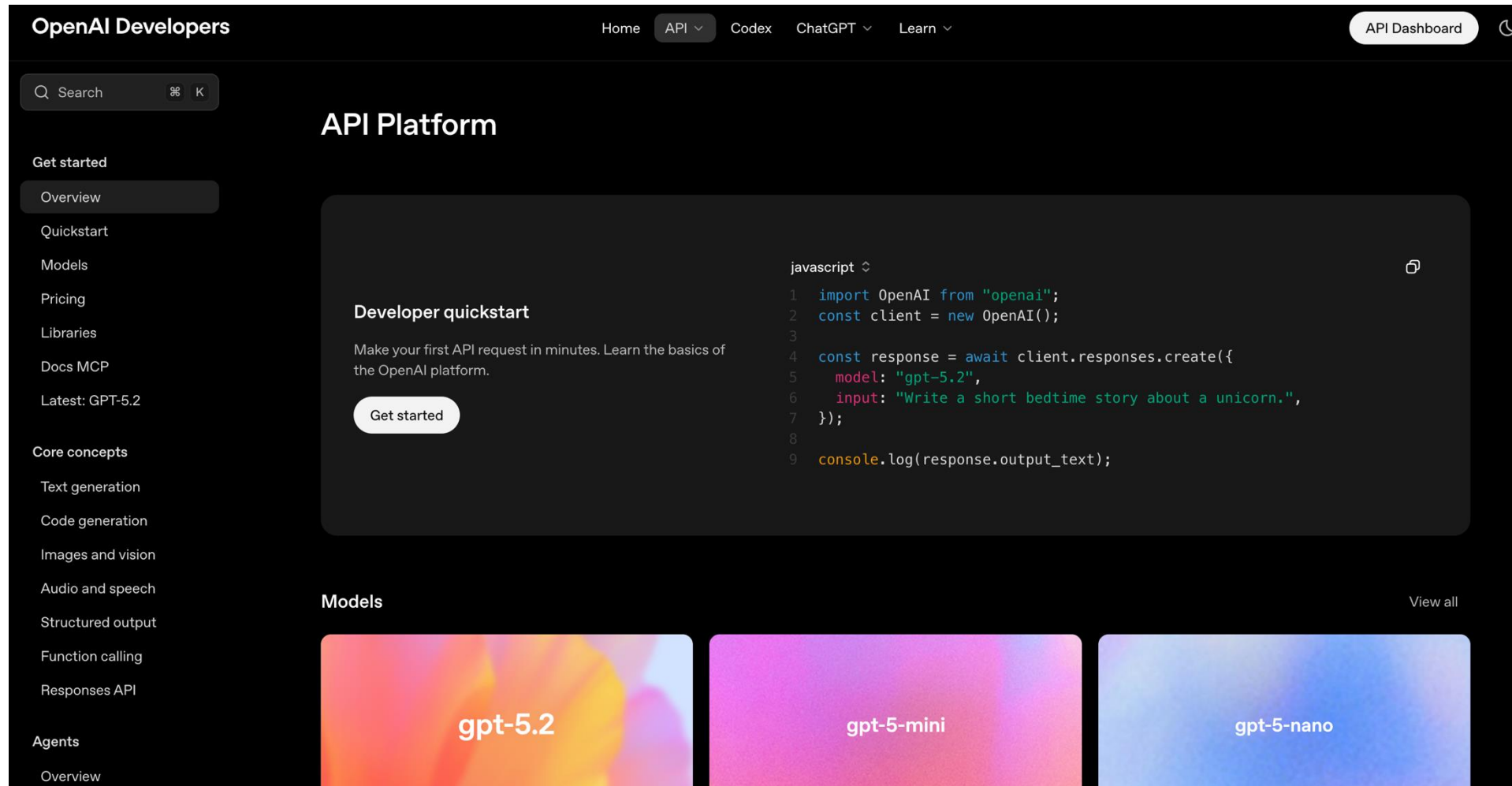
3. You receive outputs

- Assistant text (and/or tool calls)
- Metadata: token usage, finish reason, latency, etc.

API mental model

LLM API Providers Leaderboard							
 OpenAI	 GPT-4	8k	88	\$37.50	26.9	0.71	0.48
 Microsoft Azure	 GPT-4	8k	88	\$37.50	15.7	0.48	0.36
 OpenAI	 GPT-4 Turbo	128k	100	\$15.00	22.7	0.59	0.39
 Microsoft Azure	 GPT-4 Turbo	128k	100	\$15.00	13.5	0.54	0.38
 OpenAI	 GPT-4 Vision	128k	100	\$15.00	24.4	0.70	0.41
 OpenAI	 GPT-3.5 Turbo	16k	60	\$0.75	57.9	0.47	0.30
 Microsoft Azure	 GPT-3.5 Turbo	16k	60	\$0.75	42.8	0.32	0.24
 OpenAI	 GPT-3.5 Turbo Instruct	4k	60	\$1.63	82.5	0.34	0.24
 Microsoft Azure	 GPT-3.5 Turbo Instruct	4k	60	\$1.63	153.7	0.58	0.48
 Amazon	 Llama 2 Chat (70B)	4k	38	\$2.10	18.6	0.42	0.29
 OctoAI	 Llama 2 Chat (70B)	4k	38	\$0.93	24.3	0.35	0.14
 Microsoft Azure	 Llama 2 Chat (70B)	4k	38	\$1.60	16.7	2.75	2.42
 Fireworks AI	 Llama 2 Chat (70B)	4k	38	\$0.90	112.6	0.15	0.12

LLM API-ChatGPT



The screenshot shows the OpenAI Developers API Platform interface. The top navigation bar includes links for Home, API (selected), Codex, ChatGPT, and Learn, along with an API Dashboard button and a moon icon. A search bar is located on the left. The main content area is titled 'API Platform' and features a 'Developer quickstart' section with a 'Get started' button. To the right of this section is a code block for JavaScript. Below the quickstart section is a 'Models' section displaying three model cards: gpt-5.2, gpt-5-mini, and gpt-5-nano. A 'View all' link is positioned to the right of the model cards. The left sidebar contains a 'Get started' section with links to Overview, Quickstart, Models, Pricing, Libraries, Docs MCP, and Latest: GPT-5.2. Below this is a 'Core concepts' section with links to Text generation, Code generation, Images and vision, Audio and speech, Structured output, Function calling, and Responses API. At the bottom of the sidebar is an 'Agents' section with an Overview link.

OpenAI Developers

Home API Codex ChatGPT Learn API Dashboard

Search % K

API Platform

Get started

- Overview
- Quickstart
- Models
- Pricing
- Libraries
- Docs MCP
- Latest: GPT-5.2

Core concepts

- Text generation
- Code generation
- Images and vision
- Audio and speech
- Structured output
- Function calling
- Responses API

Agents

- Overview

Developer quickstart

Make your first API request in minutes. Learn the basics of the OpenAI platform.

[Get started](#)

```
javascript
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-5.2",
6   input: "Write a short bedtime story about a unicorn.",
7 });
8
9 console.log(response.output_text);
```

Models [View all](#)

- gpt-5.2
- gpt-5-mini
- gpt-5-nano

<https://developers.openai.com/api/docs>

LLM API -- ChatGPT

```
import os
from openai import OpenAI

client = OpenAI(api_key=OPENAI_API_KEY)

resp = client.responses.create(
    model="gpt-5.2",
    input="Write a one-sentence bedtime story about a unicorn."
)

print(resp.output_text)
```

LLM API -- Keys

OpenAI Platform

API Docs

Start building

R

Create

Chat

Agent Builder

Audio

Images

Videos

Assistants

Manage

Usage

API keys

ChatGPT Apps

Logs

Storage

Batches

Optimize

Evaluation

Fine-tuning

API keys

+ Create new secret key

You have permission to view and manage all API keys in this project.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	STATUS	SECRET KEY	CREATED	LAST USED ⓘ	CREATED BY	PERMISSIONS
Test For COMM117	Active	sk-...hg0A	15 Feb 2026	Never	rong mu	All  

Text tokens

Prices per 1M tokens.

Batch

Flex

Standard

Priority

Model	Input	Cached input	Output
gpt-5.2	\$1.75	\$0.175	\$14.00
gpt-5.1	\$1.25	\$0.125	\$10.00
gpt-5	\$1.25	\$0.125	\$10.00
gpt-5-mini	\$0.25	\$0.025	\$2.00
gpt-5-nano	\$0.05	\$0.005	\$0.40

LLM API -- Gemini



Google AI for Developers

<https://ai.google.dev> › [gemini-api](#) › docs

Gemini API | Google AI for Developers

Input millions of tokens to **Gemini** models and derive understanding from unstructured images, videos, and documents.

Free!

API keys

To use the Gemini API, you need an API key. This page outlines how to get one.

Pricing

Free. For developers and small projects getting started with Gemini.

[More results from google.dev »](#)

<https://ai.google.dev/gemini-api/docs>

◆ Gemini API

Docs API reference

Get started

[Overview](#)

[Quickstart](#)

[API keys](#)

[Libraries](#)

[Interactions API](#)

Models

[Gemini](#)

[Gemini 3](#)

[Nano Banana](#)

[Veo](#)

[Lyria](#)

[Imagen](#)

[Embeddings](#)

[Robotics](#)

[Text-to-speech](#)

Search



English

[Get API key](#)

[Cookbook](#)

Home › Gemini API › Docs

Gemini API

The fastest path from prompt to production with Gemini, Veo, Nano Banana, and more.

[Python](#)

[JavaScript](#)

[Go](#)

[Java](#)

[C#](#)

[REST](#)

```
from google import genai

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="Explain how AI works in a few words",
)

print(response.text)
```

[Start building](#)

Follow our Quickstart guide to get an API key and make your first API call in minutes.

LLM API-- Keys

```
▶ from google import genai

import os

os.environ["GEMINI_API_KEY"] = API_KEY
# The client gets the API key from the environment variable `GEMINI_API_KEY`.
client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview", contents="Explain how AI works in a few words"
)
print(response.text)

... AI analyzes data to identify patterns and make predictions.
```

Free!

<https://ai.google.dev/gemini-api/docs>

Prompting structure: a reliable template

- **System:** role + rules
("You are a tutor...", "Be concise...", "Use JSON...")
- **User:** task + inputs + context
("Given this text... do X...")
- **Constraints:** length, style, allowed sources, refusal conditions
- **Output format:** exact schema/template
(JSON keys, bullet count, table columns)
- **Checks/guardrails:** "If info missing, ask 1 question" / "If uncertain, say so"

```
from google import genai
from google.genai import types

client = genai.Client()

SYSTEM = """You are a helpful tutor.
Rules:
- If the user's request is missing key info, ask exactly ONE clarifying question.
- Otherwise answer directly in <= 5 bullet points.
"""

USER = "Help me optimise inference for my model deployment."

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=[
        types.Content(role="system", parts=[types.Part(text=SYSTEM)]),
        types.Content(role="user", parts=[types.Part(text=USER)]),
    ],
    config=types.GenerateContentConfig(temperature=0.3),
)

print(response.text)
```

What is the specific model architecture you are using (e.g., an LLM, a computer vision model)

Hugging Face: core building blocks



It's a place where people share ai models and research. Very free, very open source.

Libraries...

- Transformers: model + tokenizer + generation
- Datasets: data loading/processing
- Accelerate: distributed + mixed precision
- PEFT: LoRA/adapters for efficient fine-tuning

```
import os
import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, get_linear_schedule_with_warmup
from torch.utils.data import DataLoader
from accelerate import Accelerator
from peft import LoraConfig, get_peft_model, TaskType
```

- <https://huggingface.co/>

Hugging Face: core building blocks



```
import os
import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, get_linear_schedule_with_warmup
from torch.utils.data import DataLoader
from accelerate import Accelerator
from peft import LoraConfig, get_peft_model, TaskType
```

```
model_name = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token # GPT2 has no pad by default
```

```
model = AutoModelForCausalLM.from_pretrained(model_name)

lora_cfg = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    target_modules=["c_attn", "c_proj"], # GPT2 attention/projection modules
)
```

Transformer

LORA

Hugging Face generation parameters



When an LLM generates text, at each step it chooses the **next token**. Hugging Face lets you control **how it chooses**.

- **Length:** `max_new_tokens`: maximum tokens to generate

- **Determinism vs creativity:**

Greedy decoding (most deterministic): no sampling, always pick highest probability token `do_sample=False`

Sampling (more varied): choose probabilistically:

`do_sample=True`

`temperature`: lower = more conservative, higher = more random

`top_p` (nucleus sampling): sample from smallest set of tokens whose total prob $\geq p$

`top_k`: sample from top k most likely tokens

- **Repetition control:**

Discourages repeating tokens `repetition_penalty`

Prevents repeating n-grams (e.g., phrases) `no_repeat_ngram_size`

Takeaway: decoding settings change *style/variance*, not the model's knowledge.

Hugging Face generation parameters



When an LLM generates text, at each step it chooses the **next token**. Hugging Face lets you control **how it chooses**.

- **Length:** `max_new_tokens`: maximum tokens to generate

- **Determinism vs creativity:**

Greedy decoding (most deterministic): no sampling, always pick highest probability token `do_sample=False`

Sampling (more varied): choose probabilistically:

`do_sample=True`

`temperature`: lower = more conservative, higher = more random

`top_p` (nucleus sampling): sample from smallest set of tokens whose total prob $\geq p$

`top_k`: sample from top k most likely tokens

- **Repetition control:**

Discourages repeating tokens `repetition_penalty`

Prevents repeating n-grams (e.g., phrases) `no_repeat_ngram_size`

Takeaway: decoding settings change *style/variance*, not the model's knowledge.

vLLM



Efficient Memory Management for Large Language Model Serving with PagedAttention

- vLLM is an **open-source library** designed for fast, easy-to-use, and cost-efficient **LLM inference and serving**
- vLLM includes both **an inference server** (which manages network traffic), and **an inference engine** (to maximize computational speed).

<https://github.com/vllm-project/vllm?tab=readme-ov-file>

What is an inference server

- An **inference server** is **software** that helps an AI model **make new conclusions** based on its **prior training**.
- Inference servers feed **the input requests** through a machine learning model and get output

- **Inference engine**: **the low-level runtime** that executes the neural net efficiently (kernels, KV cache, batching, quantization).
- **Inference server**: **the service layer that exposes the model over a network/API** (routing, auth, rate limits, batching, logging, scaling).
Examples: vLLM **server**
- **ChatGPT**: **an end-user application (UI + conversation management)** that calls model inference via OpenAI infrastructure. It includes:

ChatGPT is an app built on top of an inference server/engine stack; the engine runs the model, the server exposes it, and ChatGPT adds UI + orchestration + safety + tools.

LLMs “Reason” with Math — Why Inference Efficiency Matters

- **Training:** the model learns by optimizing a loss function (lots of math).
- **Inference (generation):** each next word/token is chosen via **probability calculations**.

LLMs don’t “understand” like humans; they model **relationships between words** using numerical patterns.

- At scale (millions of users), inference means **huge volumes of computation** in real time.
- The main bottleneck is often **memory** (especially the **KV cache**) and **latency** under concurrency.

vLLM improves serving efficiency with architecture optimizations that reduce memory pressure and increase throughput (faster responses, more users per GPU).

KV cache: why it matters

KV cache (Key–Value cache) is a **memory store** used during Transformer **inference**.

In self-attention, each layer creates **Q, K, V** (Query, Key, Value) vectors.

- During generation, the **new token** produces a new **Q**.
- The model needs **K and V for all previous tokens** to compute attention.
- So **we cache K and V** from earlier tokens and reuse them.

Without KV cache:

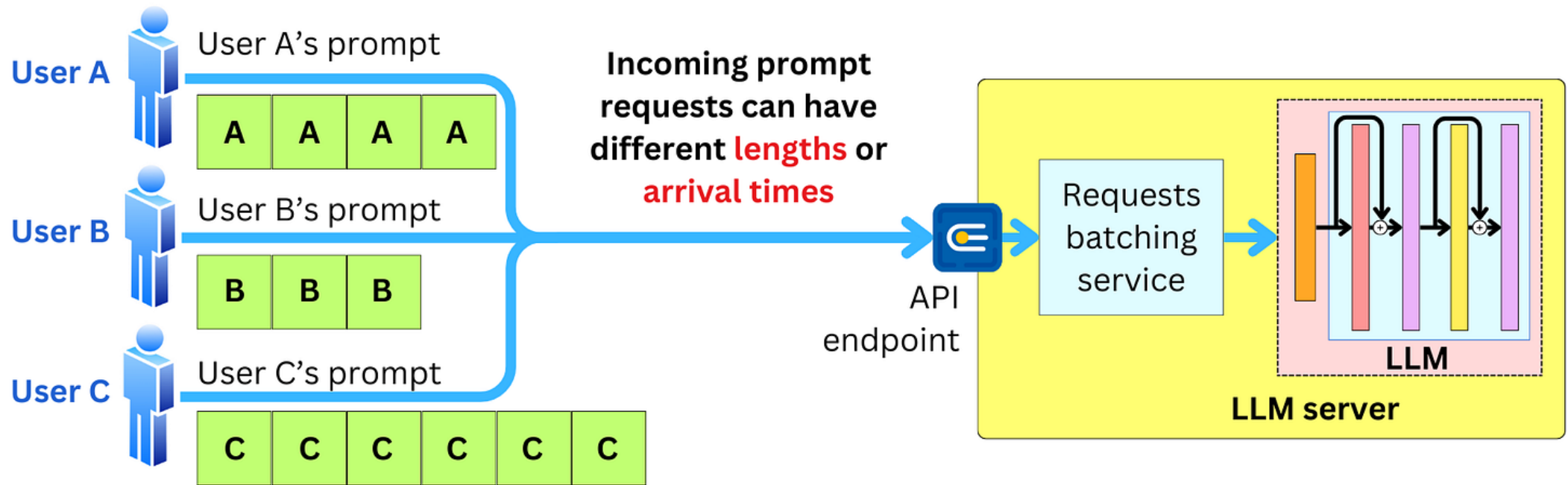
- For token 1...T, every new token would redo attention over all past tokens → **slow**.

With KV cache:

- **Past K/V are reused** → each step only computes K/V for the new token.
- **Big speedup**, especially for long contexts and multi-turn chat. **More GPU memory use!**

Batching strategies

Batching = processing multiple prompts (or decoding steps) together in one GPU forward pass.



Batching strategies

1. Static batching

- Collect N requests, then **run them together as one batch**.
- Simple, but you may have to **wait** for the batch to fill.

Good for: offline jobs / predictable traffic

Downside: increases user waiting time (latency)



2. Dynamic batching (most common in production)

- **Continuously adds** incoming requests into batches as they arrive.
- Set a time window; processes whatever requests have arrived in that time frame.

Good for: real-time apps with variable traffic

Downside: needs careful limits (otherwise tail latency explodes)



Batching strategies

1. Static batching



2. Dynamic batching (most common in production)



3. Continuous batching / token-level batching (what vLLM is known for)

- it lets each sequence in a batch finish independently
- Then immediately replaces it with a new one

Good for: many concurrent chats

Downside: more complex scheduling, but much higher throughput

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3					
S_4	S_4	S_4					

Static/Dynamic batching

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Continuous batching

Sequences **complete at different iterations** by emitting an **end-of-sequence token (red)**, at which point new sequences are inserted.

Batching strategies

1. Static batching

- Collect N requests, then **run them together as one batch**.
- Simple, but you may have to **wait** for the batch to fill.

Good for: offline jobs / predictable traffic

Downside: increases user waiting time (latency)

Static Batching

2. Dynamic batching

- **Collecting requests until the batch is full.**
- Tries to maximize GPU usage while keeping latency acceptable.

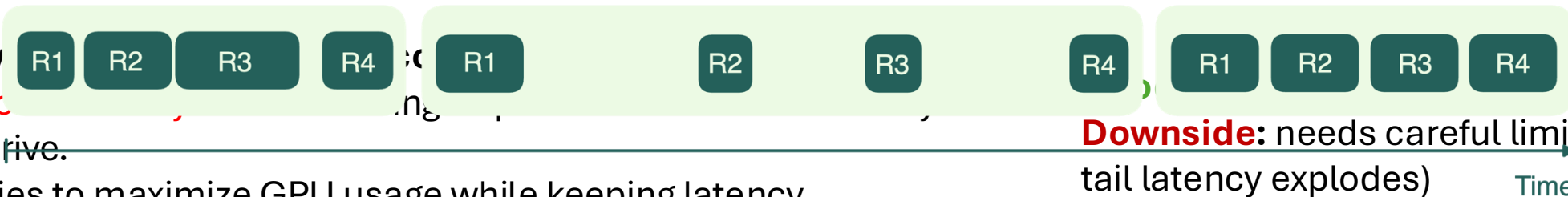
3. Continuous batching / token-level batching (what vLLM is known for)

- During decoding, requests can be at different generation steps.
- The server batches *whatever tokens are ready now* across many requests.

Downside: needs careful limits (otherwise tail latency explodes)

Good for: many concurrent chats

Downside: more complex scheduling, but much higher throughput



Checklist: making a model fast in practice

- Limit max context / truncate safely
 - Use KV cache + batching
 - Use appropriate precision (BF16/FP16 → INT8/4 if safe)
 - Use vLLM/TGI for serving
- **HF good for:** experiments, research, small-scale serving
 - **vLLM good for:** high-throughput serving, many concurrent requests, production inference

Next steps

- More about vLLM, Speculative decoding,
Text Generation: beam search, sampling, top-k nucleus sampling

Paper for next lecturer:

Efficient Memory Management for Large Language Model Serving with
Paged Attention

Additional Material:

1. [Hugging face examples](#)
2. [vLLM examples](#)

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.

 r.mu2@exeter.ac.uk

Questions?

